

Exploratory Data Analysis (EDA) is the foundation of every data science project. It is the process of examining datasets to understand their structure, identify patterns, detect anomalies and extract meaningful insights. Before applying any machine learning or statistical models, data must be cleaned, transformed and explored this is where EDA plays an important role.

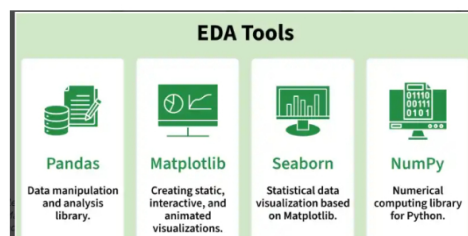
EDA helps answer important questions such as:

What type of data is present (numerical, categorical, text, dates)

Are there missing or inconsistent values

Are there outliers that could affect analysis

What patterns or relationships exist between variables



How to Perform EDA Using Python?

Step 1: Import Python Libraries

Step 2: Reading Dataset

Step 3: Data Reduction

Step 4: Feature Engineering

Step 5: Creating Features

Step 6: Data Cleaning/Wrangling

Step 7: EDA Exploratory Data Analysis

Step 8: Statistics Summary

Step 9: EDA Univariate Analysis

Step 10: Data Transformation

Step 12: EDA Bivariate Analysis

Step 13: EDA Multivariate Analysis

Step 14: Impute Missing values

Exploratory Data Analysis in Python

Double-click (or enter) to edit

Start coding or [generate](#) with AI.

Import Python Libraries

Reading Dataset from Github

```
import pandas as pd
data = 'https://raw.githubusercontent.com/makhan010385/DataSet/refs/heads/main/used_cars_data.csv'
```

Analyzing the Data

Before we make any inferences, we listen to our data by examining all variables in the data.

The main goal of data understanding is to gain general insights about the data, which covers the number of rows and columns, values in the data, datatypes, and Missing values in the dataset.

shape – shape will display the number of observations(rows) and features(columns) in the dataset

There are 7253 observations and 14 variables in our dataset

head() will display the top 5 observations of the dataset

```
data = pd.read_csv(data)
print(data.head())
```

S.No.	0	1	2	3	4	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine
0	0	0	0	0	0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	26.6 km/kg	998 CC
1	1	1	1	1	1	Hyundai Creta 1.6 CRDi SX Option	Pune	2015	44000	Diesel	Manual	First	18.5 km/kg	1493 CC
2	2	2	2	2	2	Honda Jazz V	Chennai	2011	15000	Petrol	Manual	First	18.5 km/kg	1493 CC
3	3	3	3	3	3	Maruti Ertiga VDI	Chennai	2012	25000	Diesel	Manual	First	18.5 km/kg	1493 CC
4	4	4	4	4	4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	44000	Diesel	Automatic	First	18.5 km/kg	1493 CC

```

1      44000  Diesel  Manual  First  18.2 kmpl  1199 CC
2      46000  Petrol  Manual  First  18.2 kmpl  1199 CC
3      87000  Diesel  Manual  First  20.77 kmpl  1248 CC
4      40670  Diesel  Automatic  Second  15.2 kmpl  1968 CC

```

```

      Power  Seats  New_Price  Price
0  58.16 bhp    5.0      NaN    1.75
1  126.2 bhp    5.0      NaN    12.50
2   88.7 bhp    5.0   8.61 Lakh    4.50
3  88.76 bhp    7.0      NaN    6.00
4  140.8 bhp    5.0      NaN    17.74

```

- tail() will display the last 5 observations of the dataset

```
data.tail()
```

S.No.	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	New_Price	Price
7248	Volkswagen Vento Diesel Trendline	Hyderabad	2011	89411	Diesel	Manual	First	20.54 kmpl	1598 CC	103.6 bhp	5.0	NaN	NaN
7249	Volkswagen Polo GT TSI	Mumbai	2015	59000	Petrol	Automatic	First	17.21 kmpl	1197 CC	103.6 bhp	5.0	NaN	NaN
7250	Nissan Micra Diesel XV	Kolkata	2012	28000	Diesel	Manual	First	23.08 kmpl	1461 CC	63.1 bhp	5.0	NaN	NaN
7251	Volkswagen Polo GT TSI	Pune	2013	52262	Petrol	Automatic	Third	17.2 kmpl	1197 CC	103.6 bhp	5.0	NaN	NaN
7252	Mercedes-Benz E-Class 2009-2013 E 220 CDI Avan...	Kochi	2014	72443	Diesel	Automatic	First	10.0 kmpl	2148 CC	170 bhp	5.0	NaN	NaN

data.info() shows the variables Mileage, Engine, Power, Seats, New_Price, and Price have missing values.

Numeric variables like Mileage, Power are of datatype as float64 and int64. Categorical variables like Location, Fuel_Type, Transmission, and Owner Type are of object data type.

- Check for Duplication

nunique() based on several unique values in each column and the data description, we can identify the continuous and categorical columns in the data.

Duplicated data can be handled or removed based on further analysis

```
data.nunique()
```

```

0
S.No.      7253
Name       2041
Location    11
Year        23
Kilometers_Driven  3660
Fuel_Type     5
Transmission   2
Owner_Type     4
Mileage       450
Engine        150
Power         386
Seats          9
New_Price     625
Price        1373

```

dtype: int64

Missing Values Calculation

- isnull() is widely been in all pre-processing steps to identify null values in the data

In our example, data.isnull().sum() is used to get the number of missing records in each column

```
data.isnull().sum()
```

```

0
S.No.      0
Name       0
Location    0
Year        0
Kilometers_Driven  0
Fuel_Type     0
Transmission  0
Owner_Type     0
Mileage       2
Engine        46
Power         48
Seats         53
New_Price    6247
Price        1234

```

dtype: int64

- The below code helps to calculate the percentage of missing values in each column

```
(data.isnull().sum()/(len(data)))*100
```

```

0
S.No.      0.000000
Name       0.000000
Location    0.000000
Year        0.000000
Kilometers_Driven  0.000000
Fuel_Type     0.000000
Transmission  0.000000

```

Owner_Type	0.000000
Mileage	0.027575
Engine	0.634220
Power	0.634220
Seats	0.730732
New_Price	86.129877
Price	17.013650

dtype: float64

The percentage of missing values for the columns New_Price and Price is ~86% and ~17%, respectively.

Step 3: Data Reduction

Some columns or variables can be dropped if they do not add value to our analysis.

In our dataset, the column S.No have only ID values, assuming they don't have any predictive power to predict the dependent variable.

```
# Remove S.No. column from data
data = data.drop(['S.No.'], axis = 1)
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7253 entries, 0 to 7252
Data columns (total 13 columns):
#   Column              Non-Null Count  Dtype
---  -
0   Name                 7253 non-null   object
1   Location              7253 non-null   object
2   Year                 7253 non-null   int64
3   Kilometers_Driven    7253 non-null   int64
4   Fuel_Type            7253 non-null   object
5   Transmission          7253 non-null   object
6   Owner_Type           7253 non-null   object
7   Mileage              7251 non-null   object
8   Engine               7207 non-null   object
9   Power                7207 non-null   object
10  Seats                7200 non-null   float64
11  New_Price            1006 non-null   object
12  Price                6019 non-null   float64
dtypes: float64(2), int64(2), object(9)
memory usage: 736.8+ KB
```

We start our Feature Engineering as we need to add some columns required for analysis.

Step 4: Feature Engineering

Feature engineering refers to the process of using domain knowledge to select and transform the most relevant variables from raw data when creating a predictive model using machine learning or statistical modeling.

The main goal of Feature engineering is to create meaningful data from raw data.

Step 5: Creating Features

We can play around with the variables Year and Name in our dataset. If we see the sample data, the column "Year" shows the manufacturing year of the car.

It would be difficult to find the car's age if it is in year format as the Age of the car is a contributing factor to Car Price.

Introducing a new column, "Car_Age" to know the age of the car

```
from datetime import date
date.today().year
data['Car_Age']=date.today().year-data['Year']
data.head()
```

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	New_Price	Price	Car_Age
0	Maruti Wagon R LXI CNG	Mumbai	2010	72000	CNG	Manual	First	26.6 km/kg	998 CC	58.16 bhp	5.0	NaN	1.75	16
1	Hyundai Creta 1.6 CRDI SX Option	Pune	2015	41000	Diesel	Manual	First	19.67 kmpl	1502 CC	126.2 bhp	5.0	NaN	12.50	11
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	18.2 kmpl	1199 CC	88.7 bhp	5.0	8.61 Lakh	4.50	15
3	Maruti Ertiga VDI	Chennai	2012	87000	Diesel	Manual	First	20.77 kmpl	1248 CC	88.76 bhp	7.0	NaN	6.00	14
4	Audi A4 New 2.0 TDI Multitronic	Coimbatore	2013	40670	Diesel	Automatic	Second	15.2 kmpl	1968 CC	140.8 bhp	5.0	NaN	17.74	13

Since car names will not be great predictors of the price in our current data. But we

can process this column to extract important information using brand and Model names.

Let's split the name and introduce new variables "Brand" and "Model"

```
data['Brand'] = data.Name.str.split().str.get(0)
```

```
data['Model'] = data.Name.str.split().str.get(1) + data.Name.str.split().str.get(2)
```

```
data[['Name', 'Brand', 'Model']]
```

	Name	Brand	Model
0	Maruti Wagon R LXI CNG	Maruti	WagonR
1	Hyundai Creta 1.6 CRDI SX Option	Hyundai	Creta1.6
2	Honda Jazz V	Honda	JazzV
3	Maruti Ertiga VDI	Maruti	ErtigaVDI

4	Audi A4 New 2.0 TDI Multitronic	Audi	A4New
...	...	...	...
7248	Volkswagen Vento Diesel Trendline	Volkswagen	VentoDiesel
7249	Volkswagen Polo GT TSI	Volkswagen	PoloGT
7250	Nissan Micra Diesel XV	Nissan	MicraDiesel
7251	Volkswagen Polo GT TSI	Volkswagen	PoloGT
7252	Mercedes-Benz E-Class 2009-2013 E 220 CDI Avan...	Mercedes-Benz	E-Class2009-2013

7253 rows × 3 columns

Step 6: Data Cleaning/Wrangling

Some names of the variables are not relevant and not easy to understand. Some data may have data entry errors, and some variables may need data type conversion. We need to fix this issue in the data.

In the example, The brand name 'Isuzu' 'ISUZU' and 'Mini' and 'Land' looks incorrect. This needs to be corrected

```
print(data.Brand.unique())
print(data.Brand.nunique())
```

```
['Maruti' 'Hyundai' 'Honda' 'Audi' 'Nissan' 'Toyota' 'Volkswagen' 'Tata'
 'Land' 'Mitsubishi' 'Renault' 'Mercedes-Benz' 'BMW' 'Mahindra' 'Ford'
 'Porsche' 'Datsun' 'Jaguar' 'Volvo' 'Chevrolet' 'Skoda' 'Mini' 'Fiat'
 'Jeep' 'Smart' 'Ambassador' 'Isuzu' 'ISUZU' 'Force' 'Bentley'
 'Lamborghini' 'Hindustan' 'OpelCorsa']
33
```

1 Basic Statistical Explotation

```
# Summary statistics
data.describe()
# Include categorical columns also
```

	Year	Kilometers_Driven	Seats	Price	Car_Age
count	7253.000000	7.253000e+03	7200.000000	6019.000000	7253.000000
mean	2013.365366	5.869906e+04	5.279722	9.479468	12.634634
std	3.254421	8.442772e+04	0.811660	11.187917	3.254421
min	1996.000000	1.710000e+02	0.000000	0.440000	7.000000
25%	2011.000000	3.400000e+04	5.000000	3.500000	10.000000
50%	2014.000000	5.341600e+04	5.000000	5.640000	12.000000
75%	2016.000000	7.300000e+04	5.000000	9.950000	15.000000
max	2019.000000	6.500000e+06	10.000000	160.000000	30.000000

Double-click (or enter) to edit

2 Value Counts Analysis (Very Important for Beginners)

```
# Check most common brands
data['Brand'].value_counts()
```

```
Honda      743
Toyota     507
Mercedes-Benz  380
Volkswagen  374
Ford       351
Mahindra    331
BMW        312
Audi       285
Tata       228
Skoda      202
Renault    170
Chevrolet  151
Nissan     117
Land Rover  67
Jaguar     48
Fiat       38
Mitsubishi  36
Mini Cooper  31
Volvo      28
Porsche    19
Jeep       19
Datsun     17
Isuzu      5
Force      3
Bentley    2
Ambassador  1
Smart      1
Lamborghini 1
Hindustan  1
OpelCorsa  1
```

dtype: int64

Data Type Separation

```
# Numeric columns
num_cols = data.select_dtypes(include=['int64', 'float64'])

# Categorical columns
cat_cols = data.select_dtypes(include=['object', 'category'])
```

```
cat_cols = data.select_dtypes(include=[object])
```

```
print(num_cols.columns)
print(cat_cols.columns)
```

```
Index(['Year', 'Kilometers_Driven', 'Seats', 'Price', 'Car_Age'], dtype=object)
Index(['Name', 'Location', 'Fuel_Type', 'Transmission', 'Owner_Type',
       'Mileage', 'Engine', 'Power', 'New_Price', 'Brand', 'Model'],
      dtype=object)
```

3. Correlation Analysis

```
# Correlation matrix
corr_matrix = num_cols.corr()
corr_matrix
```

	Year	Kilometers_Driven	Seats	Price	Car_Age
Year	1.000000	-0.187859	0.008216	0.305327	-1.000000
Kilometers_Driven	-0.187859	1.000000	0.090221	-0.011493	0.187859
Seats	0.008216	0.090221	1.000000	0.052225	-0.008216
Price	0.305327	-0.011493	0.052225	1.000000	-0.305327
Car_Age	-1.000000	0.187859	-0.008216	-0.305327	1.000000

Positive vs Negative correlation

Strong vs weak relation

The values range from:

Value	Meaning
+1	Perfect positive relationship
0	No relationship
-1	Perfect negative relationship

How to Read Your Output

Year, Kilometers_Driven, Seats, Price, Car_Age

Each cell tells:

If one variable increases, what happens to the other?

1 Year vs Car_Age = -1.000000

Perfect negative correlation.

2 Year vs Price = 0.305327 (Moderate Positive)

Newer cars tend to have higher price.

But not very strong → other factors also matter.

3. Car_Age vs Price = -0.305327

Older cars → lower price.

This is logical in real life.

5 Duplicate Data Checking

```
# Check duplicates
data.duplicated().sum()
print(data)
```

```
0      Maruti Wagon R LXi CNG      Mumbai 2010 \
1      Hyundai Creta 1.6 CRDi SX Option      Pune 2015
2      Honda Jazz V      Chennai 2011
3      Maruti Ertiga VDI      Chennai 2012
4      Audi A4 New 2.0 TDI Multitronic      Coimbatore 2013
...
7248      Volkswagen Vento Diesel Trendline      Hyderabad 2011
7249      Volkswagen Polo GT TSI      Mumbai 2015
7250      Nissan Micra Diesel XV      Kolkata 2012
7251      Volkswagen Polo GT TSI      Pune 2013
7252      Mercedes-Benz E-Class 2009-2013 E 220 CDI Avan...      Kochi 2014
```

```
0      72000      CNG      Manual      First      26.6 km/kg \
1      41000      Diesel      Manual      First      19.67 kmpl
2      46000      Petrol      Manual      First      18.2 kmpl
3      87000      Diesel      Manual      First      20.77 kmpl
4      40670      Diesel      Automatic      Second      15.2 kmpl
...
7248      89411      Diesel      Manual      First      20.54 kmpl
7249      59000      Petrol      Automatic      First      17.21 kmpl
7250      28000      Diesel      Manual      First      23.08 kmpl
7251      52263      Petrol      Automatic      Third      17.2 kmpl
7252      72443      Diesel      Automatic      First      10.0 kmpl
```

```
0      Engine      Power      Seats      New_Price      Price      Car_Age      Brand \
1      1592 CC      126.2 bhp      5.0      NaN      12.50      11      Hyundai
2      1199 CC      88.7 bhp      5.0      8.61 Lakh      4.50      15      Honda
3      1248 CC      88.76 bhp      7.0      NaN      6.00      14      Maruti
4      1968 CC      140.8 bhp      5.0      NaN      17.74      13      Audi
...
7248      1598 CC      103.6 bhp      5.0      NaN      NaN      15      Volkswagen
7249      1197 CC      103.6 bhp      5.0      NaN      NaN      11      Volkswagen
7250      1461 CC      63.1 bhp      5.0      NaN      NaN      14      Nissan
7251      1197 CC      103.6 bhp      5.0      NaN      NaN      13      Volkswagen
7252      2148 CC      170 bhp      5.0      NaN      NaN      12      Mercedes-Benz
```

```
0      Model
      WagonR
```

```
1      Creta1.6
2      JazzV
3      ErtigaVDI
4      A4New
...
7248   VentoDiesel
7249   PoloGT
7250   MicraDiesel
7251   PoloGT
7252   E-Class2009-2013

[7252 rows x 16 columns]
```

```
# Remove duplicates
data = data.drop_duplicates()
print(data)
```

```
0      Maruti Wagon R LXI CNG      Mumbai      2010
1      Hyundai Creta 1.6 CRDi SX Option      Pune      2015
2      Honda Jazz V      Chennai      2011
3      Maruti Ertiga VDI      Chennai      2012
4      Audi A4 New 2.0 TDI Multitronic      Coimbatore      2013
...
7248   Volkswagen Vento Diesel Trendline      Hyderabad      2011
7249   Volkswagen Polo GT TSI      Mumbai      2015
7250   Nissan Micra Diesel XV      Kolkata      2012
7251   Volkswagen Polo GT TSI      Pune      2013
7252   Mercedes-Benz E-Class 2009-2013 E 220 CDI Avan...      Kochi      2014

Kilometers_Driven Fuel_Type Transmission Owner_Type      Mileage \
0      72000      CNG      Manual      First      26.6 km/kg
1      41000      Diesel      Manual      First      19.67 kmpl
2      46000      Petrol      Manual      First      18.2 kmpl
3      87000      Diesel      Manual      First      20.77 kmpl
4      40670      Diesel      Automatic      Second      15.2 kmpl
...
7248   89411      Diesel      Manual      First      20.54 kmpl
7249   59000      Petrol      Automatic      First      17.21 kmpl
7250   28000      Diesel      Manual      First      23.08 kmpl
7251   52262      Petrol      Automatic      Third      17.2 kmpl
7252   72443      Diesel      Automatic      First      10.0 kmpl

Engine      Power      Seats      New_Price      Price      Car_Age      Brand \
0      998 CC      58.16 bhp      5.0      NaN      1.75      16      Maruti
1      1582 CC      126.2 bhp      5.0      NaN      12.50      11      Hyundai
2      1199 CC      88.7 bhp      5.0      8.61 Lakh      4.50      15      Honda
3      1248 CC      88.76 bhp      7.0      NaN      6.00      14      Maruti
4      1968 CC      140.8 bhp      5.0      NaN      17.74      13      Audi
...
7248   1598 CC      103.6 bhp      5.0      NaN      NaN      15      Volkswagen
7249   1197 CC      103.6 bhp      5.0      NaN      NaN      11      Volkswagen
7250   1461 CC      63.1 bhp      5.0      NaN      NaN      14      Nissan
7251   1197 CC      103.6 bhp      5.0      NaN      NaN      13      Volkswagen
7252   2148 CC      170 bhp      5.0      NaN      NaN      12      Mercedes-Benz

Model
0      WagonR
1      Creta1.6
2      JazzV
3      ErtigaVDI
4      A4New
...
7248   VentoDiesel
7249   PoloGT
7250   MicraDiesel
7251   PoloGT
7252   E-Class2009-2013

[7252 rows x 16 columns]
```

Shape, Columns & Memory Analysis

```
# Rows and columns
data.shape
```

```
(7252, 16)
```

```
# Column names
data.columns
```

```
Index(['Name', 'Location', 'Year', 'Kilometers_Driven', 'Fuel_Type',
       'Transmission', 'Owner_Type', 'Mileage', 'Engine', 'Power', 'Seats',
       'New_Price', 'Price', 'Car_Age', 'Brand', 'Model'],
      dtype='object')
```

```
# Memory usage
data.memory_usage(deep=True)
```

```
0
Index      58016
Name       545050
Location   405004
Year       58016
Kilometers_Driven  58016
Fuel_Type   398642
Transmission  405007
Owner_Type   392868
Mileage     423443
Engine      404292
Power       411749
Seats       58016
New_Price   258642
Price       58016
Car_Age     58016
Brand       402485
Model       417684

dtype: int64
```

This command shows how much RAM (memory) each column of your DataFrame is using.

It returns memory in bytes.

2. Sorting & Top Records Exploration

```
# Top expensive cars
data.sort_values(by='Price', ascending=False).head(10)
```

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	New_Price	Price	Car_Age	Brand	Model
4079	Land Rover Range Rover 3.0 Diesel LWB Vogue	Hyderabad	2017	25000	Diesel	Automatic	First	13.33 kmpl	2993 CC	255 bhp	5.0	2.3 Cr	160.00	9	Land Rover	RoverRange
5781	Lamborghini Gallardo Coupe	Delhi	2011	6500	Petrol	Automatic	Third	6.4 kmpl	5204 CC	560 bhp	2.0	NaN	120.00	15	Lamborghini	GallardoCoupe
5919	Jaguar F Type 5.0 V8 S	Hyderabad	2015	8000	Petrol	Automatic	First	12.5 kmpl	5000 CC	488.1 bhp	2.0	NaN	100.00	11	Jaguar	FType
1505	Land Rover Range Rover Sport SE	Kochi	2019	26013	Diesel	Automatic	First	12.65 kmpl	2993 CC	255 bhp	5.0	1.39 Cr	97.07	7	Land Rover	RoverRange
1974	BMW 7 Series 740Li	Coimbatore	2018	28060	Petrol	Automatic	First	12.05 kmpl	2979 CC	320 bhp	5.0	NaN	93.67	8	BMW	7Series
1984	BMW 7 Series 740Li	Bangalore	2017	17465	Petrol	Automatic	First	12.05 kmpl	2979 CC	320 bhp	5.0	NaN	93.00	9	BMW	7Series
4691	Mercedes-Benz SLK-Class 55 AMG	Bangalore	2014	3000	Petrol	Automatic	Second	12.0 kmpl	5461 CC	421 bhp	2.0	NaN	90.00	12	Mercedes-Benz	SLK-Class55
5535	BMW X6 xDrive 40d M Sport	Ahmedabad	2015	97003	Diesel	Automatic	First	15.87 kmpl	2993 CC	308.43 bhp	5.0	NaN	85.00	11	BMW	X6xDrive
2095	Mercedes-Benz SLC 43 AMG	Coimbatore	2019	2526	Petrol	Automatic	First	19.0 kmpl	2996 CC	362.07 bhp	2.0	1.06 Cr	83.96	7	Mercedes-Benz	SLC43
2422	Jaguar XJ 3.0L Portfolio	Delhi	2016	12000	Diesel	Automatic	First	10.5 kmpl	2993 CC	271.23 bhp	4.0	NaN	79.00	10	Jaguar	XJ3.0L

```
# Oldest cars
data.sort_values(by='Car_Age', ascending=False).head()
```

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	New_Price	Price	Car_Age	Brand	Model
6216	Hindustan Motors Contessa 2.0 DSL	Pune	1996	65000	Diesel	Manual	Second	14.1 kmpl	1995 CC	null bhp	5.0	NaN	NaN	30	Hindustan	MotorsContessa
4709	Maruti 1000 AC	Hyderabad	1998	104000	Petrol	Manual	Second	15.0 kmpl	970 CC	null bhp	5.0	NaN	0.85	28	Maruti	1000AC
3749	Mercedes-Benz E-Class 250 D W 210	Mumbai	1998	55300	Diesel	Automatic	First	10.0 kmpl	1796 CC	157.7 bhp	5.0	NaN	3.90	28	Mercedes-Benz	E-Class250
3138	Maruti Zen LXI	Jaipur	1998	95150	Petrol	Manual	Third	17.3 kmpl	993 CC	60 bhp	5.0	NaN	0.45	28	Maruti	ZenLXI
5716	Maruti Zen LX	Jaipur	1998	95150	Petrol	Manual	Third	17.3 kmpl	993 CC	60 bhp	5.0	NaN	0.53	28	Maruti	ZenLX

Filtering / Query Operations

```
# Petrol cars only
data[data['Fuel_Type'] == 'Petrol']
```

	Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	New_Price	Price	Car_Age	Brand	Model
2	Honda Jazz V	Chennai	2011	46000	Petrol	Manual	First	18.2 kmpl	1199 CC	88.7 bhp	5.0	8.61 Lakh	4.50	15	Honda	JazzV
10	Maruti Ciaz Zeta	Kochi	2018	25692	Petrol	Manual	First	21.56 kmpl	1462 CC	103.25 bhp	5.0	10.65 Lakh	9.95	8	Maruti	CiazZeta
11	Honda City 1.5 V AT Sunroof	Kolkata	2012	60000	Petrol	Automatic	First	16.8 kmpl	1497 CC	116.3 bhp	5.0	NaN	4.49	14	Honda	City1.5
22	Audi A6 2011-2015 35 TFSI Technology	Mumbai	2015	55985	Petrol	Automatic	First	13.53 kmpl	1984 CC	177.01 bhp	5.0	NaN	23.50	11	Audi	A62011-2015
23	Hyundai i20 1.2 Magna	Kolkata	2010	45807	Petrol	Manual	First	18.5 kmpl	1197 CC	80 bhp	5.0	NaN	1.87	16	Hyundai	i201.2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
7245	Honda Amaze S i-Vtech	Kochi	2015	44776	Petrol	Manual	First	18.0 kmpl	1198 CC	86.7 bhp	5.0	NaN	NaN	11	Honda	AmazeS
7246	Hyundai Grand i10 AT Asta	Coimbatore	2016	18242	Petrol	Automatic	First	18.9 kmpl	1197 CC	82 bhp	5.0	NaN	NaN	10	Hyundai	Grandi10
7247	Hyundai EON D Lite Plus	Coimbatore	2015	21190	Petrol	Manual	First	21.1 kmpl	814 CC	55.2 bhp	5.0	NaN	NaN	11	Hyundai	EOND
7249	Volkswagen Polo GT TSI	Mumbai	2015	59000	Petrol	Automatic	First	17.21 kmpl	1197 CC	103.6 bhp	5.0	NaN	NaN	11	Volkswagen	PoloGT
7251	Volkswagen Polo GT TSI	Pune	2013	52262	Petrol	Automatic	Third	17.2 kmpl	1197 CC	103.6 bhp	5.0	NaN	NaN	13	Volkswagen	PoloGT

3324 rows × 16 columns

```
# Multiple condition
data[(data['Car_Age'] < 5) & (data['Price'] > 10)]
```

Name	Location	Year	Kilometers_Driven	Fuel_Type	Transmission	Owner_Type	Mileage	Engine	Power	Seats	New_Price	Price	Car_Age	Brand	Model
------	----------	------	-------------------	-----------	--------------	------------	---------	--------	-------	-------	-----------	-------	---------	-------	-------

Double-click (or enter) to edit

outlier detection methods

```
import pandas as pd
import numpy as np

data = pd.DataFrame({'Marks':[45,50,52,48,49,300]})
data
```

Marks
0 45
1 50
2 52
3 48
4 49
5 300

```
Q1 = data['Marks'].quantile(0.25)
Q3 = data['Marks'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR

outliers_iqr = data[(data['Marks'] < lower) | (data['Marks'] > upper)]
outliers_iqr
```

Colab paid products - Cancel contracts here



Variables

Terminal